

DAA – TOPIC 11

1. Bellman-Ford Algorithm

- **Aim:** Find shortest distances from a source vertex and detect a negative-weight cycle.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def bellman_ford(vertices, edges, source):
    dist = [float("inf")] * vertices
    dist[source] = 0
    for _ in range(vertices - 1):
        for u, v, w in edges:
            if dist[u] != float("inf") and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] != float("inf") and dist[u] + w < dist[v]:
            return None
    return dist

edges=[(0,1,-1),(0,2,4),(1,2,3),(1,3,2),(1,4,2),
        (3,2,5),(3,1,1),(4,3,-3)]
print(bellman_ford(5,edges,0))
```

OUTPUT

Output

```
[0, -1, 2, -2, 1]
```

- **Conclusion:** The algorithm obtains the shortest distances and can detect negative cycles.

2. Floyd-Warshall Algorithm

- **Aim:** Find shortest distances between every pair of vertices in a weighted graph.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def floyd_warshall(graph):
    d=[row[:] for row in graph]
    n=len(d)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    return d

INF=float("inf")
g=[[0,5,INF,10],
   [INF,0,3,INF],
   [INF,INF,0,1],
   [INF,INF,INF,0]]
print(*floyd_warshall(g),sep="\n")
```

OUTPUT

Output

```
[0, 5, 8, 9]\n[INF, 0, 3, 4]\n[INF, INF, 0, 1]\n[INF, INF, INF, 0]
```

- **Conclusion:** Floyd-Warshall computes all-pairs shortest paths correctly.

3. Warshall's Algorithm

- **Aim:** Compute the transitive closure of a directed graph.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def warshall(g):
    n=len(g)
    r=[row[:] for row in g]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                r[i][j]=r[i][j] or (r[i][k] and r[k][j])
    return r

g=[[0,1,0,0],
   [0,0,1,0],
   [0,0,0,1],
   [0,0,0,0]]
print(*warshall(g),sep="\n")
```

OUTPUT

Output

```
[0, 1, 1, 1]\n[0, 0, 1, 1]\n[0, 0, 0, 1]\n[0, 0, 0, 0]
```

- **Conclusion:** Warshall's algorithm identifies whether a path exists between every pair of vertices.

4. City with the Smallest Number of Reachable Cities

- **Aim:** Use Floyd-Warshall to find the city reaching the fewest other cities within a distance threshold.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def find_city(n, edges, threshold):
    INF=10**9
    d=[[INF]*n for _ in range(n)]
    for i in range(n): d[i][i]=0
    for u,v,w in edges:
        d[u][v]=d[v][u]=w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    counts=[sum(d[i][j]<=threshold for j in range(n) if i!=j)
            for i in range(n)]
    return min(range(n), key=lambda i:(counts[i],-i))

edges=[[0,1,3],[1,2,1],[2,3,4],[0,3,7]]
print(find_city(4,edges,4))
```

OUTPUT

Output

3

- **Conclusion:** The city with the fewest reachable cities within the threshold is 3.

5. Network Delay Time

- **Aim:** Determine the time required for a signal to reach all nodes using Bellman-Ford relaxation.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def network_delay(times,n,k):
    INF=10**9
    d=[INF]*(n+1); d[k]=0
    for _ in range(n-1):
        changed=False
        for u,v,w in times:
            if d[u]!=INF and d[u]+w<d[v]:
                d[v]=d[u]+w; changed=True
        if not changed: break
    return max(d[1:]) if max(d[1:])<INF else -1

times=[[2,1,1],[2,3,1],[3,4,1]]
print("Network Delay =",network_delay(times,4,2))
```

OUTPUT

Output

Network Delay = 2

- **Conclusion:** The signal reaches every node, and the total delay is 2 units.

6. Cheapest Flights Within K Stops

- **Aim:** Find the cheapest route from a source to a destination with at most K stops.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def cheapest_flight(n, flights, src, dst, k):
    INF=10**9
    d=[INF]*n; d[src]=0
    for _ in range(k+1):
        nd=d[:]
        for u,v,w in flights:
            if d[u]!=INF:
                nd[v]=min(nd[v],d[u]+w)
        d=nd
    return d[dst] if d[dst]<INF else -1

flights=[[0,1,100],[1,2,100],[0,2,500]]
print("Cheapest Cost =",cheapest_flight(3,flights,0,2,1))
```

OUTPUT

Output

Cheapest Cost = 200

- **Conclusion:** The cheapest flight within the allowed stop limit costs 200.

7. Find Negative Cycle in Directed Graph

- **Aim:** Detect whether a weighted directed graph contains a negative-weight cycle.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def has_negative_cycle(n, edges):
    d=[0]*n
    for _ in range(n-1):
        for u,v,w in edges:
            d[v]=min(d[v],d[u]+w)
    for u,v,w in edges:
        if d[u]+w<d[v]:
            return True
    return False

edges=[(0,1,1),(1,2,-1),(2,0,-1)]
print("Negative Cycle:",has_negative_cycle(3,edges))
```

OUTPUT

Output

Negative Cycle: True

- **Conclusion:** The extra relaxation after $n-1$ iterations confirms a negative-weight cycle.

8. All-Pairs Shortest Path

- **Aim:** Compute shortest distances for every pair of vertices using Floyd-Warshall.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def all_pairs(g):
    d=[r[:] for r in g]
    n=len(d)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    return d
```

```
INF=float("inf")
g=[[0,2,INF],[INF,0,4],[1,INF,0]]
print(*all_pairs(g),sep="\n")
```

OUTPUT

Output

```
[0, 2, 6]\n[5, 0, 4]\n[1, 3, 0]
```

- **Conclusion:** The algorithm produces the shortest path matrix for all vertex pairs.

9. Transitive Closure of a Directed Graph

- **Aim:** Determine reachability between all pairs of vertices using Warshall's method.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def transitive_closure(g):
    n=len(g); r=[row[:] for row in g]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                r[i][j]=r[i][j] or (r[i][k] and r[k][j])
    return r

g=[[0,1,1],[0,0,1],[1,0,0]]
print(*transitive_closure(g),sep="\n")
```

OUTPUT

Output

```
[1, 1, 1]\n[1, 1, 1]\n[1, 1, 1]
```

- **Conclusion:** Every vertex is reachable from every other vertex in the supplied graph.

10. Minimum Distance Between All Cities

- **Aim:** Calculate the minimum distances between all cities using a weighted adjacency matrix.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def minimum_distances(g):
    d=[r[:] for r in g]
    n=len(d)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    return d

INF=float("inf")
g=[[0,5,INF,10],[INF,0,3,INF],
   [INF,INF,0,1],[INF,INF,INF,0]]
print(*minimum_distances(g),sep="\n")
```

OUTPUT

Output

```
[0, 5, 8, 9]\n[INF, 0, 3, 4]\n[INF, INF, 0, 1]\n[INF, INF, INF, 0]
```

- **Conclusion:** The minimum distances between all city pairs are obtained using dynamic programming.

11. Shortest Path with Negative Edges

- **Aim:** Find shortest paths from a source when some edge weights are negative.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def shortest_path(n, edges, source):
    INF=10**9; d=[INF]*n; d[source]=0
    for _ in range(n-1):
        for u,v,w in edges:
            if d[u]!=INF:
                d[v]=min(d[v],d[u]+w)
    return d

edges=[(0,1,4),(0,2,5),(1,2,-2),(2,3,3)]
print(shortest_path(4,edges,0))
```

OUTPUT

Output

```
[0, 4, 2, 5]
```

- **Conclusion:** Bellman-Ford handles negative edges and returns the correct shortest distances.

12. Reachability in a Directed Network

- **Aim:** Determine whether every vertex can reach every other vertex by computing transitive closure.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def reachable(g):
    n=len(g); r=[row[:] for row in g]
    for i in range(n): r[i][i]=1
    for k in range(n):
        for i in range(n):
            for j in range(n):
                r[i][j]=r[i][j] or (r[i][k] and r[k][j])
    return r

g=[[0,1,0],[0,0,1],[1,0,0]]
print(*reachable(g),sep="\n")
```

OUTPUT

Output

```
[1, 1, 1]\n[1, 1, 1]\n[1, 1, 1]
```

- **Conclusion:** The closure shows that all vertices are mutually reachable.

13. Shortest Routes in a Road Network

- **Aim:** Find shortest routes between all locations in a road network using Floyd-Warshall.

- **Algorithm:**

1. Initialize the required graph or cost table.
2. Process vertices and edges using the selected dynamic-programming recurrence.
3. Update the current shortest-path or reachability state.
4. Repeat until all relevant states are processed.
5. Display the final result.

PROGRAM

Python Program

```
def road_routes(g):
    d=[r[:] for r in g]; n=len(d)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    return d

INF=float("inf")
g=[[0,4,INF,10],[4,0,3,INF],
   [INF,3,0,2],[10,INF,2,0]]
print(*road_routes(g),sep="\n")
```

OUTPUT

Output

```
[0, 4, 7, 9]\n[4, 0, 3, 5]\n[7, 3, 0, 2]\n[9, 5, 2, 0]
```

- **Conclusion:** The shortest road distance between every pair of locations is computed successfully.